



>> ARM 프로세서 특징

✓ 명령어 Instructions

- 축약된 명령어 세트/단일사이클/고정길이 명령어 Reduced instruction sets/single cycle/fixed length

✓ 파이프라인 Pipeline

- 1 사이클에 해석됨/ 마이크로코드 불필요 Decoded in one stage/no need for microcode

✓ 레지스터 Registers

- 많은 수의 범용 목적 레지스터 Large set of general-purpose registers



>> ARM 프로세서 특징

✓ 로드/스토어 아키텍처 Load/store architecture

- 데이터 처리 명령이 레지스터만을 대상으로 함 Data processing instructions applied to registers only
- 로드/스토어 명령어는 메모리와 데이터간 데이터를 전송함 Load/store instructions transfers data from memory to registers

✓ 명령어 파이프라인

- 5단계 파이프라인 5-stage pipeline

>> ARM Cortex-M3 vs. ARM7TDMI

특징 Features	ARM7TDMI-S	Cortex-M3
아키텍처 Architecture	ARMv4T(von Neumann)	ARMv7-M(Harvard)
명령어세트 지원 ISA Support	Thumb / ARM	Thumb / Thumb-2
파이프라인 Pipeline	3-Stage	3-Stage + branch speculation
인터럽트 Interrupts	FIQ / IRQ	NMI + 1 to 240 Physical Interrupts
인터럽트 지연시간 Interrupt Latency	24-42 Cycles	12 Cycles
슬립 모드 Sleep Modes	None	Integrated
메모리 보호 Memory Protection	None	8 region Memory Protection Unit
드라이스톤 Dhrystone	0.95 DIMPS/MHz (ARM mode)	1.25 DIMPS/MHz
전력 소모 Power Consumption	0.28 mW/MHz	0.19 mW/MHz
Area	0.62 mm ² (Core Only)	0.86 mm ² (Core & Peripherals)*

>> ARM 프로세서 특권모드

모드 Mode	축약형 Abbreviation	특권모드 Privileged	비트[4:0] Bits[4:0]
Abort	Abt	Yes	10111
Fast Interrupt	Fiq	Yes	10001
Interrupt request	Irq	Yes	10010
Supervisor	Svc	Yes	10011
System	Sys	Yes	11111
Undefined	Und	Yes	11011
User	Usr	No	10000

>> 임베디드 개발환경

☑ FreeRTOS

- FreeRTOS는 로열티 걱정이 없고 메모리 사용량이 적은 실시간 운영체제임
FreeRTOS is a royalty-free mini Real-Time Kernel with a very small footprint
- <http://www.freertos.org> 사이트에서 무료로 다운로드 가능 Free download – www.freertos.org
- 30개 이상의 아키텍처에 포팅되어 있고, 17개 툴체인이 제공됨 Ported to 30+ architectures and 17 toolchains
- 많은 아키텍처와 통합개발환경을 대상으로 예제 프로젝트 존재 Example projects for many architectures and IDEs

>> 임베디드 개발환경

☑ μ C/OS-II or III

- 마이크로프로세서를 위한 우선순위 기반 실시간 멀티태스킹 운영체제 커널 Priority-based real-time multitasking operating system kernel for microprocessors
- 매우 적은 메모리 사용량 Very small footprint
- 다양한 보드를 대상으로 보드 서포트 패키지(board support package) 제공 Board Support Package for various board
- 리눅스 Cortex-M 릴리즈 버전 1.4 Linux Cortex-M Release 1.4

ARM GPIO의 개념과 사용법

>> GPIO

- ✓ 이러한 문제를 해결하기 위하여, multiple function 기능을 사용하면 각 GPIO 포트가 그 설정에 따라 다양한 기능을 수행할 수 있음 To solve this problem, signal multiplexing allows each single GPIO to get one of several p

GPIO	1st function	2nd function	3rd function	4th function	MCU pin
P5	P0[9]	I ² S (TX_SDA)	SSP1 (MOSI)	MAT2[3]	76
P6	P0[8]	I ² S (TX_WS)	SSP1 (MISO)	MAT2[2]	77
P7	P0[7]	I ² S (TX_CLK)	SSP1 (SCK)	MAT2[1]	78
P8	P0[6]	I ² S (RX_SDA)	SSP1 (SSEL)	MAT2[0]	79
P9	P0[0]	CAN1 (RD1)	UART3 (TX)	I ² C1 (SDA)	46
P10	P0[1]	CAN1 (TD1)	UART3(RX)	I ² C1 (SCL)	47
P11	P0[18]	UART1 (DCD)	SSP0 (MOSI)	SPI (MOSI)	60
P12	P0[17]	UART1 (CTS)	SSP0 (MISO)	SPI (MISO)	61
P13	P0[15]	UART1 (TXD)	SSP0 (SCK)	SPI (SCK)	62
P14	P0[16]	UART1 (RXD)	SSP0 (SSEL)	SPI (SSEL)	63
P15	P0[23]	A/D0[0]	I ² S (RX_CLK)	CAP3[0]	9
P16	P0[24]	A/D0[1]	I ² S (RX_WS)	CAP3[1]	8
P17	P0[25]	A/D0[2]	I ² S (RX_SDA)	UART3 (TXD)	7
P18	P0[26]	A/D0[3]	DAC out	UART3 (RXD)	6
P19	P1[30]	V _{BUS}	A/D0[4]		21
P20	P1[31]	SSP1 (SCK)	A/D0[5]		20
P21	P2[5]	PWM1[6]	UART1 (DTR)	TRACEDATA[0]	68
P22	P2[4]	PWM1[5]	UART1 (DSR)	TRACEDATA[1]	69
P23	P2[3]	PWM1[4]	UART1 (DCD)	TRACEDATA[2]	70
P24	P2[2]	PWM1[3]	UART1 (CTS)	TRACEDATA[3]	73
P25	P2[1]	PWM1[2]	UART1 (RXD)		74
P26	P2[0]	PWM1[1]	UART1 (TXD)		75
P27	P0[11]	UART2 (RXD)	I ² C2 (SCL)	MAT3[1]	49
P28	P0[10]	UART2 (TXD)	I ² C2 (SDA)	MAT3[0]	48
P29	P0[5]	I ² S (RX_WS)	CAN2 (TD)	CAP2[1]	80
P30	P0[4]	I ² S (RX_CLK)	CAN2 (RD)	CAP2[0]	81
D+	P0[29]	USB (D+)			29
D-	P0[30]	USB (D-)			30
RX+	(Ethernet)				-
RX-					
TX+					
TX-					

>> GPIO 핀 설명 Pin description

✓ GPIO 핀 설명 GPIO pin description

Pin Name	Type	Description
P0[30:0]; P1[31:0]; P2[13:0]; P3[26:25]; P4[29:28]	Input/ Output	범용목적 I/O, 전형적으로 다른 주변장치 function 들과 함께 공유되며, 한 응용프로그램에서 사용가능하지 않을 수 있음. 패키징 옵션은 특정 디바이스에서 사용가능한 GPIO의 수에 영향을 미칠 수 있다. 일부 핀들은 해당 핀의 alternate functio의 요구사항에 의해서 제한될 수 있다. 예를 들어, I2C0핀은 그 핀에 선택된 어떠한 동작에 대해서도 open-drain 상태이다. Packaging options may affect the number of GPIOs available in a particular device. Some pins may be limited by requirements of the alternate functions of the pin. For example, the pins containing the I^2C0 functions are open-drain for any function selected on that pin.

>> 입력 처리 (인터럽트)

✓ EXTINT

- 인터럽트 발생시 EINT0부터 EINT3까지 한 비트가 세트되고 적절한 코드가 수행을 시작함 Once a bit from EINT0 to EINT3 is set and an appropriate code starts to execute
- EINT0부터 EINT3 레지스터의 한 비트에 1값을 쓰는 것은 그에 해당하는 인터럽트를 클리어하는 것과 동일한 효과 Writing ones to bits EINT0 through EINT3 in EXTINT register clears the corresponding



인터럽트 처리

>> 입력 처리 (인터럽트)

✓ EXTMODE 레지스터

Bit	Symbol	Value	Description	Reset value
0	EXTMODE0	0	Level-sensitivity is selected for $\overline{\text{EINT0}}$	0
		1	$\overline{\text{EINT0}}$ is edge sensitive	
1	EXTMODE1	0	Level-sensitivity is selected for $\overline{\text{EINT1}}$	0
		1	$\overline{\text{EINT1}}$ is edge sensitive	
2	EXTMODE2	0	Level-sensitivity is selected for $\overline{\text{EINT2}}$	0
		1	$\overline{\text{EINT2}}$ is edge sensitive	
3	EXTMODE3	0	Level-sensitivity is selected for $\overline{\text{EINT3}}$	0
		1	$\overline{\text{EINT3}}$ is edge sensitive	
7:4	-	-	예약됨. 사용자 소프트웨어는 예약된 비트에 값을 쓰지 않아야 하며, 또한 예약된 비트로부터 값을 읽는 경우 그 결과는 신뢰할 수 없음. Reserved, user software should not write ones to reserved bits. The value read from a reserved bit is not defined.	NA



인터럽트 처리

>> 입력 처리 (인터럽트)

✓ EXTPOLAR 레지스터

Bit	Symbol	Value	Description	Reset value
0	EXTPOLAR0	0	$\overline{\text{EINT0}}$ is low-active or falling-edge sensitive (depending on EXTMODE0)	0
		1	$\overline{\text{EINT0}}$ is high-active or falling-edge sensitive (depending on EXTMODE0)	
1	EXTPOLAR1	0	$\overline{\text{EINT1}}$ is low-active or falling-edge sensitive (depending on EXTMODE1)	0
		1	$\overline{\text{EINT1}}$ is high-active or falling-edge sensitive (depending on EXTMODE1)	

7차시

>> 타이머

Generic Name	Description	Access	Reset value ^[1]	PORTn Register Address & Name
IR	인터럽트 레지스터 Interrupt Register. The IR can be written to clear interrupts. The IR can be read to identify which of eight possible interrupt sources are pending.	R/W	0	T0IR – 0x4000 4000 T1IR – 0x4000 8000 T2IR – 0x4009 0000 T3IR – 0x4009 4000
TCR	타이머 컨트롤 레지스터 Timer Control Register. The TCR is used to control the Timer Counter functions. The Timer Counter can be disabled or reset through the TCR.	R/W	0	T0TCR – 0x4000 4004 T1TCR – 0x4000 8004 T2TCR – 0x4009 0004 T3TCR – 0x4009 4004
TC	타이머 카운터 Timer counter. The 32-bit TC is incremented every PR+1 cycles of PCLK. The TC is controlled through the TCR.	R/W	0	T0TC – 0x4000 4008 T1TC – 0x4000 8008 T2TC – 0x4009 0008 T3TC – 0x4009 4008

ARM LPC1768의 타이머 사용방법

>> 타이머

Generic Name	Description	Access	Reset value ^[1]	PORTn Register Address & Name
PR	프리스케일 레지스터 Prescale Register. The Prescale Counter(below) is equal to this value, the next clock increments the TC and clears the PC.	R/W	0	T0PR – 0x4000 400C T1PR – 0x4000 800C T2PR – 0x4009 000C T3PR – 0x4009 400C
PC	프리스케일 카운터 Prescale Counter. The 32-bit PC is a counter which is incremented to the value stored in PR. When the value in PR is reached, the TC is incremented and the PC is cleared. The PC is observable and controllable through the bus interface.	R/W	0	T0PC – 0x4000 4010 T1PC – 0x4000 8010 T2PC – 0x4009 0010 T3PC – 0x4009 4010
MCR	매치 컨트롤 레지스터 Match Control Register. The MCR is used to control if an interrupt is generated and if the TC is reset when a Match occurs.	R/W	0	T0MCR – 0x4000 4014 T1MCR – 0x4000 8014 T2MCR – 0x4009 0014 T3MCR – 0x4009 4014

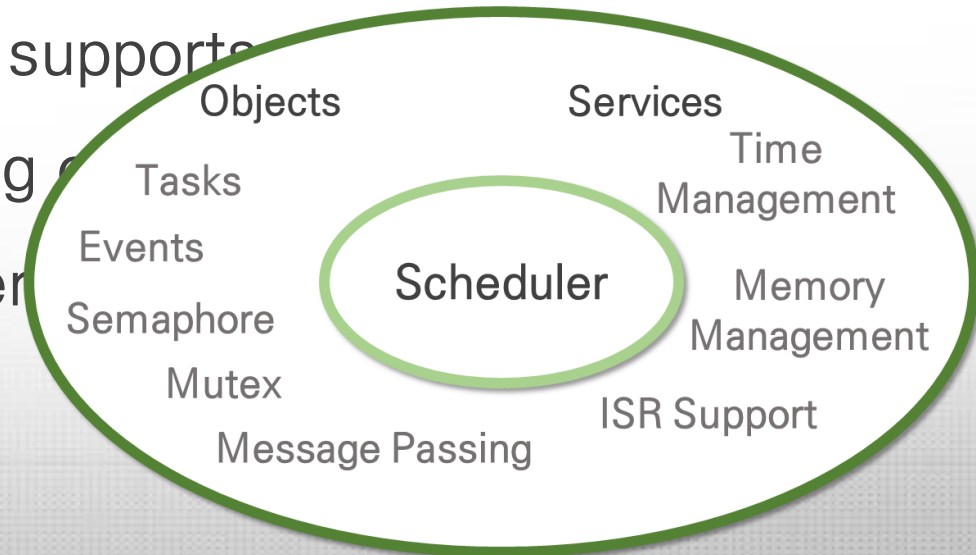
>> 타이머

Generic Name	Description	Access	Reset value ^[1]	PORTn Register Address & Name
MR0	매치 레지스터 0 Match Register 0. MR0 can be enabled through the MCR to reset the TC, stop both the TC and PC, and/or generate an interrupt every time MR0 matches the TC.	R/W	0	T0MR0 – 0x4000 4018 T1MR0 – 0x4000 8018 T2MR0 – 0x4009 0018 T3MR0 – 0x4009 4018
MR1	매치 레지스터 1 Match Register 1. See MR0 description	R/W	0	T0MR1 – 0x4000 401C T1MR1 – 0x4000 801C T2MR1 – 0x4009 001C T3MR1 – 0x4009 401C
MR2	매치 레지스터 2 Match Register 2. See MR0 description	R/W	0	T0MR2 – 0x4000 4020 T1MR2 – 0x4000 8020 T2MR2 – 0x4009 0020

>> RTOS

☑ CMSIS-RTOS

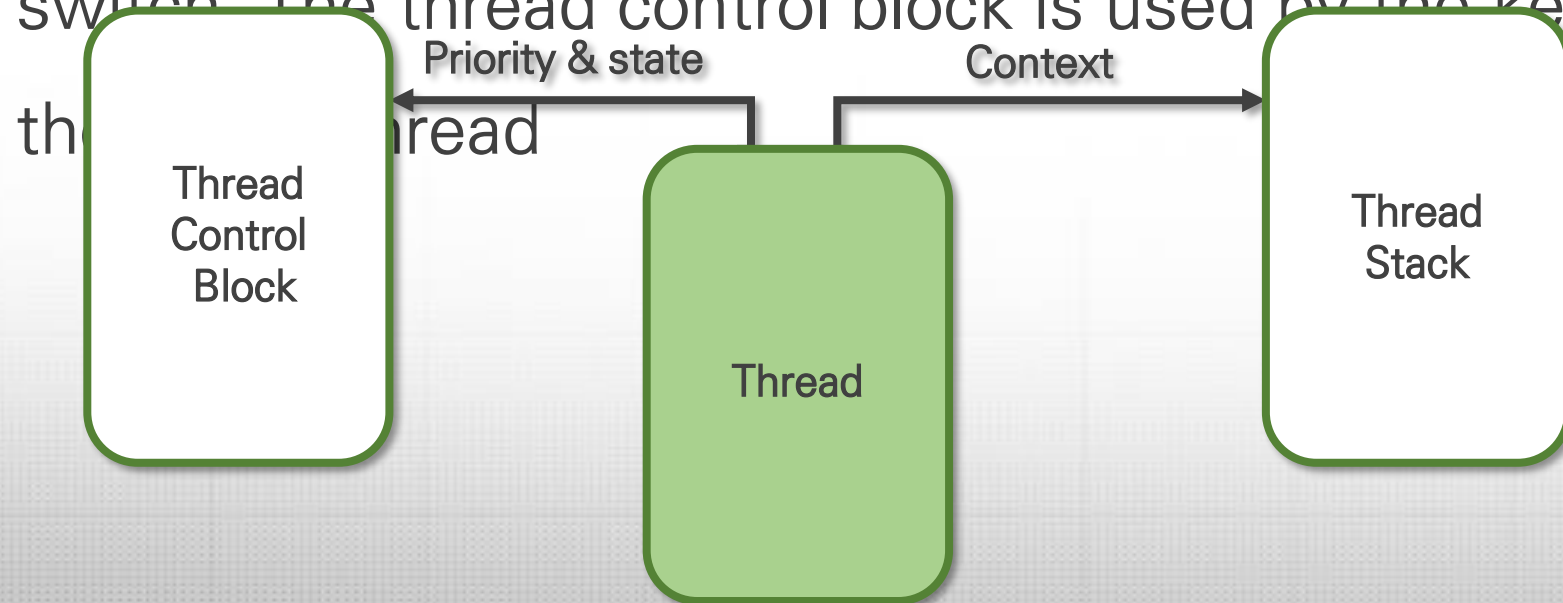
- RTOS는 스케줄러를 포함하는데, 라운드 로빈, pre-emptive/co-operative한 프로그램 쓰레드의 멀티태스킹을 지원하고, 시간 및 메모리 관리 서비스를 제공함.
- RTOS itself consists of a scheduler which supports pre-emptive and co-operative multitasking as well as time and memory management services.



>> RTOS 쓰레드

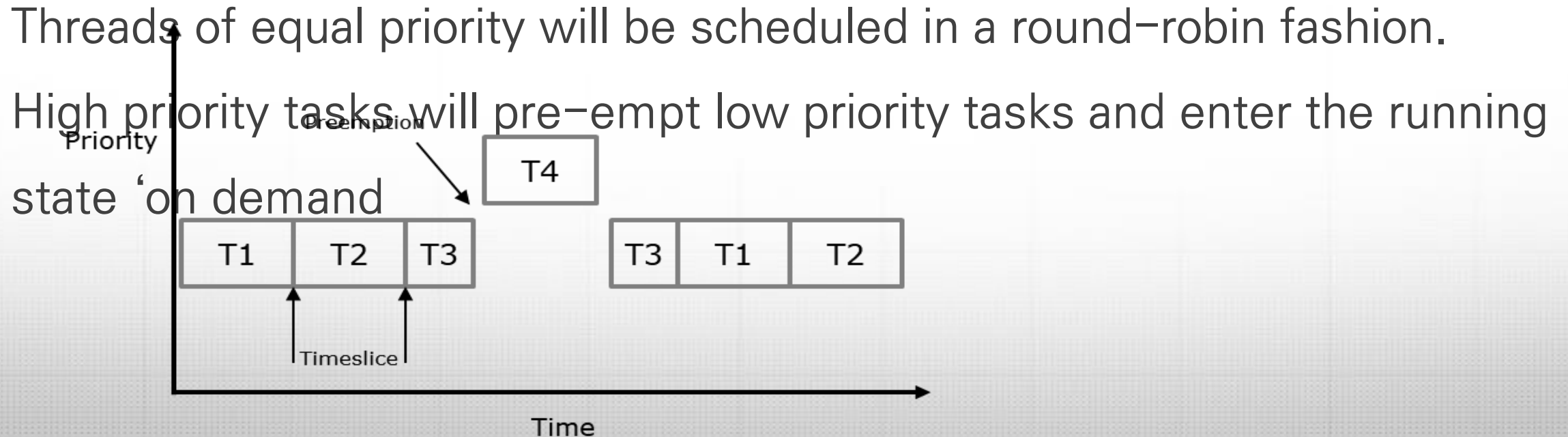
- ✓ 각 쓰레드는 컨텍스트 교체시 보관하여야 하는 자신만의 스택을 가짐
- ✓ 쓰레드 컨트롤 블록은 커널이 액티브한 쓰레드를 관리하는데 사용됨

Each thread has its own stack for saving its data during a context switch. The thread control block is used by the kernel to manage the thread.



>> RTOS 쓰레드

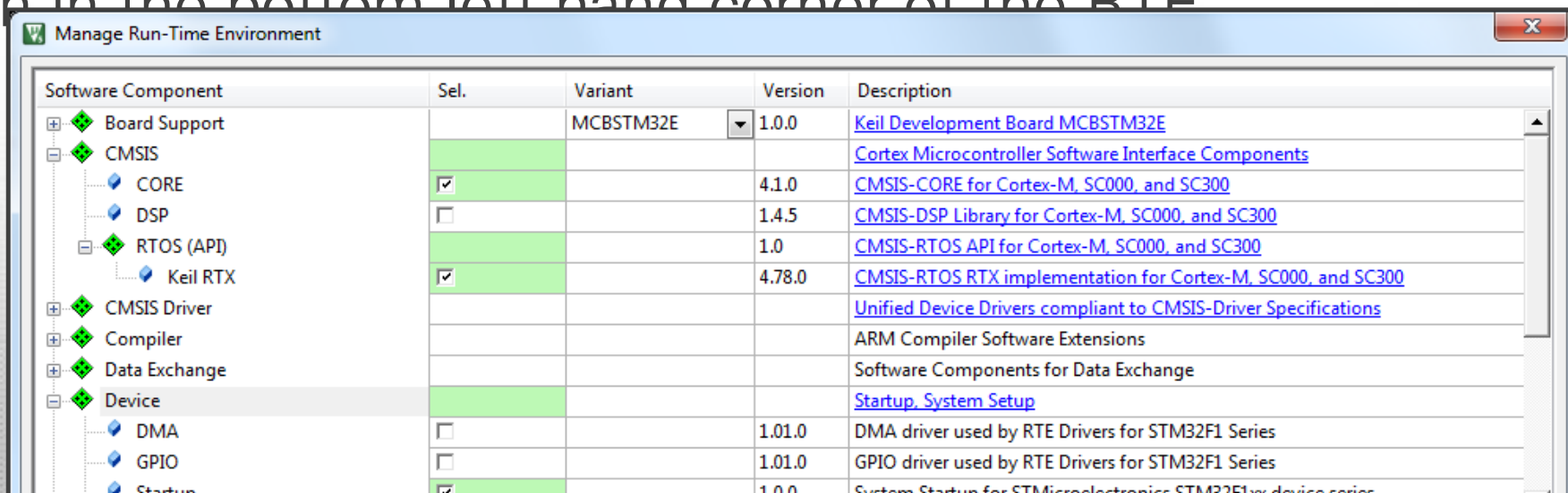
- ✓ 동일한 우선순위를 가지는 쓰레드들은 라운드 로빈 방식으로 스케줄 됨
- ✓ 높은 우선순위 태스크는 낮은 우선순위의 태스크를 선점하고 자신이 CPU를 할당받아 실행상태로 진입할 수 있음



>> RTOS 프로젝트 설정

- ✓ 실행시간 환경 관리(Manage Run-Time Management 메뉴에서 Resolve 버튼을 누름으로써, 의존성 관계에 있으나 현재 누락되어 있는 컴포넌트를 자동으로 추가할 수 있음

To add the missing components you can press the “Resolve” button in the bottom left hand corner of the RTE



>> RTOS 다중 쓰레드 (Blocked wait)

- ✓ 쓰레드는 수행시간 동안 많은 상태에 걸쳐 변화함. 실행하는 쓰레드는 osDelay() 함수호출에 의해서 wait 상태로 들어가면서 블럭킹할 수 있음.
- ✓ osDelay() 함수에서 지정한 대기 시간이 경과하면, 그 쓰레드는 ready 상태로 전이함으로써, 스케줄러가 추후에 run 상태로 dispatch할 수 있도록 한다.
- ✓ 만약 해당 쓰레드에 할당된 타임슬라이스가 소진되면, 다시 ready 상태로

복귀한다.

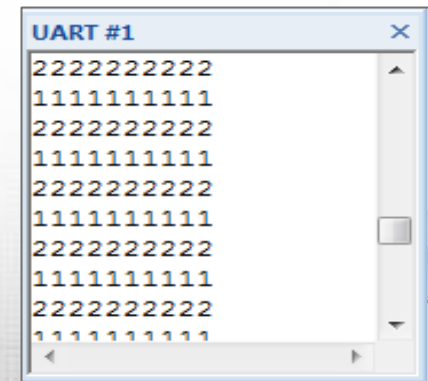
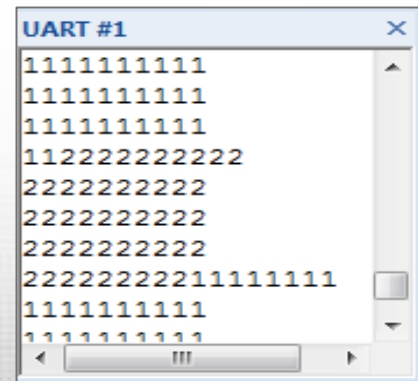
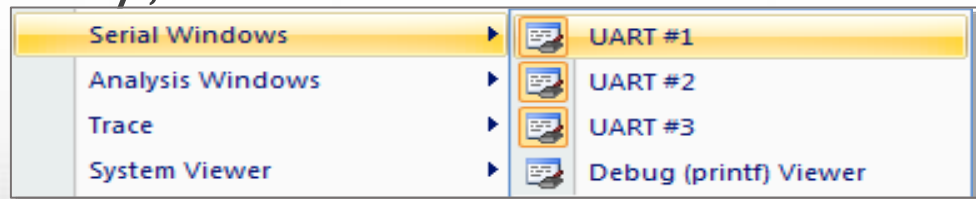
During their lifetime threads move through many states. Here a running thread is blocked by an osDelay call so it enters a wait state. When the delay expires, it moves to ready. The scheduler will place it in the run

>> RTOS 쓰레드에서 동기화

- 이 프로젝트는 UART에 문자를 출력하는 두개 쓰레드를 동시에 수행하는 경우 각 쓰레드가 출력하는 행위를 서로 동기화 하기 위해서 쓰레드 동기화 기법이 사용됨.

declares two threads which both write blocks of characters to the **UART**.

Initially, the mutex is commented out.



>> KEIL μ Vision MDK의 디버깅 기능

- ✓ Access Break (A), Execution Break (E), Conditional Break (C)

Access Break (A)	특정한 메모리 접근이 발생되었을때 프로그램의 실행을 정지함. Defined when the flags Read, or Write, or both are set. The breakpoint is triggered when the specified memory access occurs.
Execution Break (E)	특정한 코드 주소에 도달(실행)하였을 때, 프로그램의 실행을 정지함. Defined when Expression resolves to a code address. The breakpoint triggers when the specified code address is reached.
Conditional Break (C)	사전에 정해진 조건식이 TRUE로 평가되었을 때, 해당 문장의 실행을 정지함. Defined when Expression cannot be reduced to an address. The breakpoint triggers when the conditional

>> KEIL μ Vision MDK의 디버깅 기능

✓ 메모리 윈도우 Memory

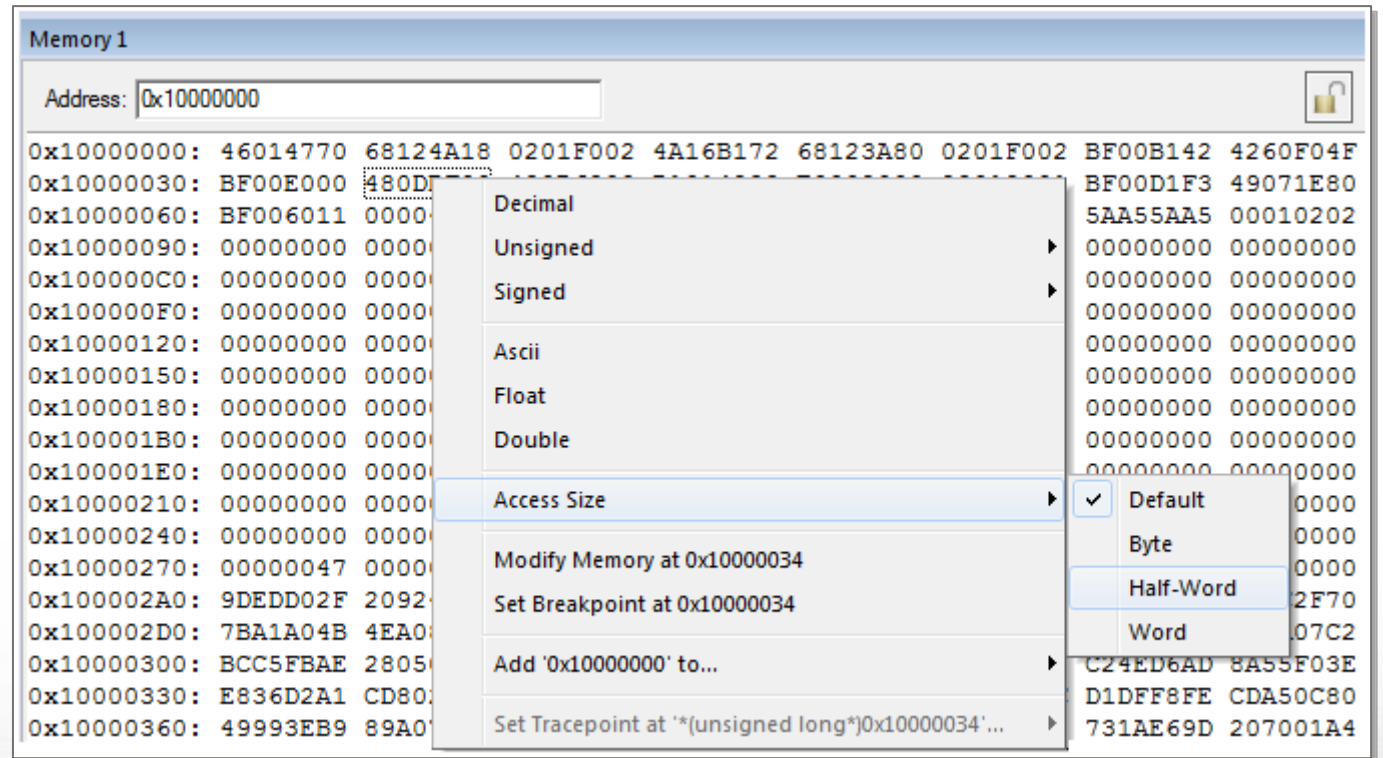
Window

- 메모리 영역의 내용을 표시함

Displays the memory area content.

- 여러개의 메모리 윈도우가 동시에 표시될 수 있음

Several memory windows can be used at a time.



>> Memory Dump

Black

코드 메모리 영역 또는 초기화 되지 않은 RAM

CODE memory area or uninitialized RAM.

Red

Flash 또는 ROM에 저장된 CONST 데이터로서 최소 한번 접근된 바 있는 내용

Gold

CONST data in Flash or ROM that has been accessed at least once.

초기화된 메모리, 그러나 아직 접근되지 않은 영역 Memory location

Green

that has been initialized, but not accessed yet.

최소한 한번 접근된 메모리 영역

Memory location has been accessed at least once.

Execution Profiler

- ✓ 프로그램의 실행이 종료되기 까지, 명령어에 대한 타이밍과 실행 통계자료를 기록함 Records timing and execution statistics about instructions for the complete program code

